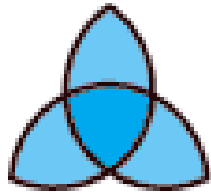


SOUTH EAST EUROPEAN UNIVERSITY

Faculty of Contemporary Sciences and Technologies



**UNIVERSITETI I ETL
JHE YHMBEP3MTET
SEE UNIVERSITY**

SDP4J

Design and Implementation of a Schema-First Lightweight Persistence Framework for Java

Capstone Project

Candidate: Amar Jahiji

Mentor: Prof. Jaumin Ajdari

BSc in Computer Sciences

Tetovo, 2026

Abstract

Java backend applications have two dominant ways of talking to a relational database: JDBC, the low-level API that ships with the platform, and a full object-relational mapping (ORM) framework such as Hibernate or Jakarta Persistence (JPA). JDBC is simple in principle but forces the developer to repeat the same boilerplate — opening a connection, preparing a statement, binding positional parameters, walking a result set and mapping columns to fields by hand. ORMs remove this boilerplate but replace it with a large new layer of concepts (persistence contexts, lazy-loading proxies, dirty checking, caches and a query language of their own) that must be learned on top of Java and SQL, and that is often unjustified for small and medium projects.

This thesis proposes SDP4J (Simple Data Persistence for Java), a schema-first, lightweight persistence framework for small-to-medium Java projects targeting PostgreSQL. Its guiding principle is that the relational schema is authoritative and should be made visible rather than hidden, so that a developer who already knows Java and SQL needs no third large concept to use it. The framework rests on three pillars: annotation-based mapping whose annotations correspond one-to-one with real SQL features, a code-driven migration engine that derives the always-safe additive changes automatically by comparing the mapped classes against the live database while requiring destructive changes to be declared explicitly, executing the whole operation atomically under an advisory lock and a two-layer query interface offering a low-level named-parameter JDBC wrapper and a high-level, schema-validated fluent builder.

The result is a working framework, validated by an automated unit-and-integration test suite and a set of runnable examples, that demonstrates the schema-first model end to end. Compared with raw JDBC it removes the repetitive resource-management and mapping code, and compared with Hibernate it removes the standing conceptual overhead, while covering mapping, migration and querying in a single tool. The migration engine is the work's principal original contribution: it occupies a gap between fully manual migration tools and unsafe automatic schema generation.

Table of Contents

Abstract.....	1
Table of Acronyms.....	3
Table of Figures.....	4
Table of Tables.....	4
1. Introduction.....	5
1.1 Background and Motivation.....	5
1.2 Problem Statement.....	6
1.3 Proposed Solution.....	6
1.4 Importance of Work.....	7
1.5 Expected Results.....	8
1.6 Structure of the Document.....	8
2. Related Work.....	9
2.1 Heavyweight ORMs and the Critique of Their Complexity.....	9
2.2 Lighter Java Persistence Tools.....	10
2.3 The Industry Shift Back to Simpler Persistence.....	10
2.4 Database Migration Tools.....	11
2.5 Cross-Language Comparators.....	11
3. Methodology.....	12
3.1 From Problem to Design Constraints.....	12
3.2 Technology and Platform Choices.....	12
3.3 System Architecture.....	13
3.4 The Schema-First Data Model.....	14
3.5 Migration Methodology.....	16
3.6 Query Methodology.....	17
3.7 End-to-End Solution Flow.....	17
3.8 Data Acquisition and Handling.....	19
3.9 Validation Methodology.....	19
4. Implementation and Results.....	21
4.1 Project Structure and Build.....	21
4.2 The Client.....	21
4.3 Annotation Mapping.....	21
4.4 DDL Generation.....	22
4.5 The Migration Engine.....	22
4.6 The Low-Level Layer (SPS4J).....	23
4.7 The High-Level Layer (SQ4J).....	23
4.8 Validation and Results.....	24
4.8.1 Behavioural comparison with other tools.....	24
4.8.2 Conceptual surface.....	26
4.8.3 Migration-safety experiment.....	27
4.8.3 Test coverage.....	28
4.9 Discussion.....	29
5. Conclusions.....	30
5.1 Achievements.....	30
5.2 Limitations.....	30
5.3 Future Work.....	31
6. Bibliography.....	32

Table of Acronyms

Acronym	Meaning
SDP4J	Simple Data Persistence for Java (framework)
SM4J	Schema Migration for Java (migration module)
SQ4J	Simple Query for Java (high-level query module)
SPS4J	Simple Prepared Statement for Java (low-level module)
API	Application Programming Interface
AST	Abstract Syntax Tree
CLI	Command-Line Interface
CRUD	Create, Read, Update, Delete
DAO	Data Access Object
DDL	Data Definition Language
DSL	Domain-Specific Language
DTO	Data Transfer Object
HQL	Hibernate Query Language
JDBC	Java Database Connectivity
JPA	Jakarta Persistence API (formerly Java Persistence API)
JPQL	Jakarta Persistence Query Language
ORM	Object-Relational Mapping
SQL	Structured Query Language
TOCTOU	Time-Of-Check to Time-Of-Use (a class of race condition)

Table of Figures

Figure	Meaning
Figure 1.	System architecture of SDP4J
Figure 2.	End-to-end developer workflow

Table of Tables

Table	Name
Table 1.	SDP4J mapping annotations
Table 2.	Java-to-PostgreSQL type mapping
Table 3.	Concepts required for basic CRUD across persistence tools
Table 4.	Outcome of renaming a column on a populated table
Table 5.	Unit-test coverage (35 tests, no database)
Table 6.	Integration-test coverage (17 tests, real PostgreSQL)

1. Introduction

1.1 Background and Motivation

Every Java backend that talks to a relational database must bridge two worlds: the object model of the program and the table model of the database. The standard low-level answer is JDBC, which gives the developer direct control over connections, statements, result sets and transactions and leaves everything else to the application. In practice this produces a great deal of repetition: a single read requires opening a connection, preparing a statement, binding parameters by position, executing the query, walking the result set, mapping each column to a field by hand, and closing resources in the correct order. A mid-sized backend accumulates hundreds of such blocks, each one an opportunity to leak a connection or to introduce SQL injection through string concatenation.

To remove this burden the ecosystem developed ORM frameworks, of which Hibernate — the reference implementation of Jakarta Persistence — is the most widely used. Hibernate maps annotated classes to tables and then manages SQL generation, change tracking and transactions automatically. The productivity gain is real, but it comes with a large body of framework-specific concepts: persistence contexts, entity lifecycle states, lazy-loading proxies, dirty checking, first- and second-level caches, flush modes and a query language of its own. Using Hibernate well requires book-length references, and even its own maintainers document that the `LazyInitializationException` is among the most common errors its users meet.

A long-running critique in the practitioner literature captures the cost. Neward famously called ORM “the Vietnam of computer science,” and Eder, the creator of jOOQ, has argued for a decade that ORMs train developers to think in object graphs and to hide SQL, when SQL is an expressive language that does not need hiding. The point is not that Hibernate is poorly built, it is that it tries to do everything at once, and that breadth is rarely justified for smaller projects. Lighter Java tools exist — jOOQ for type-safe SQL, MyBatis for SQL mapping, Jdbi as a thin JDBC wrapper — but each solves only one slice of the problem, so a small project still ends up combining several tools, plus a separate migration tool such as Flyway or Liquibase.

1.2 Problem Statement

The Java persistence landscape does not currently offer a single tool that fits small and medium projects well. JDBC is too low-level: technically simple but practically exhausting. Hibernate is the opposite: it removes the boilerplate but adds a third large concept on top of Java and SQL. The lighter tools each handle one concern well but force the developer to assemble several pieces. Beyond this gap, the dominant ORM approach exhibits recurring frictions: the same record is represented three times (row, managed entity, DTO) and must be kept in sync, repository methods are generated from long method names that fail at runtime, transactional annotations can flush in-memory changes without an explicit save, and relationship mapping forces a careful choice among several strategies. Each is small in isolation, but together they demand significant ongoing investment from every developer who uses the framework.

1.3 Proposed Solution

SDP4J is proposed as a third option between JDBC and the heavyweight ORMs. Its guiding principle is that the relational schema is authoritative and the framework should make it visible rather than hide it. The framework covers mapping, migration and querying in one dependency, with minimal setup, so a developer can move from an empty project to a working data layer in a few steps. It is a framework — it drives schema generation, migration, mapping and query execution on the developer's behalf — distributed as an ordinary Java library, a single JAR dependency. Its entry point is a client object, initialised either from a JDBC URL (with sensible default pooling) or from a caller-supplied `DataSource`.

Architecturally it rests on three pillars. The first is annotation-based mapping: a class is annotated to describe its table and each field to describe its column, with annotations that name real SQL features (primary keys, nullability, uniqueness, foreign keys and their on-delete rules, indexes, defaults) rather than inventing new vocabulary, column types are inferred from the Java field type. The second, and the most distinctive, is a code-driven migration engine that treats the annotated classes as the authoritative description of the schema and separates changes by how dangerous they are — deriving the always-safe additive changes automatically, while requiring destructive or altering changes to be declared

explicitly — and applies them atomically. The third is a two-layer query interface: a low-level wrapper over JDBC that removes its repetitive parts, and a high-level fluent builder for the common cases whose method names mirror SQL.

1.4 Importance of Work

The work is important both practically and conceptually. Practically, there is clear and recent industry demand for lighter Java persistence: Spring introduced Spring Data JDBC in 2018 as a deliberate simplification of Spring Data JPA, Micronaut Data computes queries at compile time to avoid runtime reflection, and schema-first tools such as Prisma have grown quickly in other ecosystems. SDP4J contributes to this trend by combining the schema-first philosophy with annotation conventions familiar to Java developers and by integrating migration management into the framework rather than treating it as a separate concern.

Conceptually, the work engages with the object-relational impedance mismatch, traditionally answered either by building a more comprehensive ORM or by abandoning objects. SDP4J takes a less common middle path: accept the mismatch as real, expose it, and provide a thin and clear bridge across it. Within this stance, the migration engine is the concrete point at which the thesis claims an original contribution. The established positions are fully manual versioned migrations (Flyway, Liquibase) and fully automatic schema derivation (Hibernate's auto-update); the first is tedious and drifts from the code, the second is convenient but unsafe because it will attempt destructive changes unbidden. SDP4J shows the choice need not be binary: by deriving only the provably-safe additive changes, forcing destructive ones to be explicit, and relying on transactional DDL and an advisory lock for atomicity and concurrency, a framework can be both convenient and safe at once.

safely: additive changes, which dominate the day-to-day evolution of a schema, are automated, while the comparatively rare destructive changes stay explicit and therefore remain self-documenting in version control.

1.5 Expected Results

The expected output is a working Java library, distributable through Maven, that supports schema declaration through annotations, automatic schema generation and ordered migrations, and a unified two-layer query interface against PostgreSQL, accompanied by example applications and an automated test suite. The expected qualitative outcome is that SDP4J substantially reduces persistence-related code relative to raw JDBC while keeping SQL behaviour transparent and predictable in a way Hibernate does not. The quantitative expectations are deliberately bounded: SDP4J does not aim to outperform Hibernate on every benchmark or to match jOOQ on dialect coverage. Its success is measured by how quickly a developer reaches a working data layer covering all three concerns, and by how readable and predictable the resulting code is. Accordingly, the comparison in Chapter 4 is realised structurally — code volume and the number of distinct concepts a developer must hold — rather than as a runtime performance benchmark, which is identified as future work.

1.6 Structure of the Document

Chapter 2 surveys related work — the heavyweight ORMs and the critique of their complexity, the lighter Java tools, the recent industry shift toward simpler persistence, the dedicated migration tools, and comparable frameworks in other languages — and locates SDP4J within that landscape. Chapter 3 presents the methodology: the design constraints derived from the problem, the technology and platform choices, the system architecture, the schema-first data model, the migration and query methodologies, the end-to-end solution flow, the handling of data, and the validation approach. Chapter 4 describes the implementation in steps with the essential code, then reports the validation and a structural comparison against JDBC and Hibernate, with discussion. Chapter 5 draws conclusions and specifies future work. Chapter 6 lists the references.

2. Related Work

This chapter reviews the work SDP4J is built in conversation with, grouped into five themes: the heavyweight ORMs and the critique of their complexity, the lighter Java persistence tools, the recent industry shift toward simpler persistence, the dedicated migration tools, and

comparable frameworks from other language ecosystems. For each source it states briefly what the tool does, the problem it addresses, and why it matters for SDP4J.

2.1 Heavyweight ORMs and the Critique of Their Complexity

Hibernate, described in its user guide and in *Java Persistence with Hibernate* by Bauer, King and Gregory, maps classes to tables, manages a persistence context with automatic dirty checking, supports lazy loading through proxies, provides first- and second-level caches, and exposes its own query language, HQL. It is the reference implementation of Jakarta Persistence, whose specification defines entities, the `EntityManager`, JPQL, the Criteria API and lifecycle callbacks. The framework is powerful and well documented, but the sheer volume of documentation signals how much must be learned before using it well, Mihalcea's *High-Performance Java Persistence* runs to several hundred pages precisely because the defaults often need tuning.

A substantial body of practitioner writing examines the costs. Neward's 2006 essay coined the most influential metaphor in the debate, framing ORM as a quagmire that traps its users, Atwood amplified the argument in favour of treating the database, rather than the object graph, as the model. Eder, the creator of jOOQ, has argued the same point from a SQL-centric angle across many articles, and Mihalcea — himself a Hibernate committer — documents that the `LazyInitializationException` is among the most common errors users meet, with a DTO projection usually the right fix. Each source confirms a different facet of one argument: Hibernate is not bad, but it is heavy, and that weight is often unjustified for small and medium projects.

2.2 Lighter Java Persistence Tools

Beneath the heavyweight ORMs, several Java tools each handle one part of the problem. jOOQ is the strongest tool for type-safe SQL: a code generator reads a live schema and produces Java classes, and a fluent DSL enforces SQL grammar at compile time, a jOOQ query reads almost exactly like the SQL it produces. Its limitation, for a project that needs all concerns covered, is that it focuses on querying and leaves migrations and an opinionated

mapping layer to other tools. SDP4J takes inspiration from jOOQ's fluent style for its high-level layer.

MyBatis pairs hand-written SQL with Java mapper interfaces, declaring SQL either in XML files or in annotation strings, it is honest about SQL but its XML configuration is a real source of friction and, like jOOQ, it solves only one concern. Jdbi is a deliberately minimal wrapper over JDBC offering a fluent API, named-parameter binding and pluggable mappers, SDP4J's low-level layer shares its spirit, but Jdbi does not attempt mapping or migration. Two further tools are relevant: Querydsl provides type-safe query construction over JPA and SQL through generated query types, and Ebean is a JPA-compatible ORM that rejects the session model in favour of a session-less, immediate-save API with built-in migration tooling — the closest cousin to SDP4J's ambition, though its JPA-annotation compatibility places it nearer the heavyweight end.

2.3 The Industry Shift Back to Simpler Persistence

Two recent projects confirm a broad move back toward simpler persistence. Spring Data JDBC, introduced in 2018, was presented by its lead developer as a deliberate simplification of JPA: loading and saving issue SQL immediately, with no session, no dirty tracking and no proxies, and one repository per aggregate root. That the Spring team built it inside their own ecosystem is itself a statement that the heavyweight approach is not always right, SDP4J shares its immediate-execution philosophy. Micronaut Data, from 2019, makes a related but architecturally different argument: it computes queries at build time, eliminating runtime reflection and dynamic proxies, with concrete benefits in startup, memory and validation. Its lesson for SDP4J is that schema metadata can be processed ahead of time, although SDP4J does not currently pursue that level of compilation.

2.4 Database Migration Tools

Schema migration is usually a separate concern, and the existing tools cluster at two ends of a spectrum that SDP4J's engine is designed to sit between. At the manual end, Flyway uses versioned plain-SQL or Java scripts tracked in a history table and applied in order, exactly once each, it deliberately rejects automatic down-migrations because destructive changes cannot be reversed reliably. The cost of this safety is that every change is written by hand, including purely additive ones that merely restate the entity classes. Liquibase is more

flexible — changesets in XML, YAML, JSON or SQL, with automatic inverse statements for rollback — but for small projects adds configuration and another file format, and it too requires every change to be authored explicitly.

At the automatic end, Hibernate's schema generation derives changes by comparing the mapped entities against the database. This removes the manual labour but is widely regarded as unsafe in production because it derives all changes the same way and cannot be trusted with destructive operations. SDP4J's engine occupies the gap: like Hibernate it derives changes automatically, but only the always-safe additive ones, like Flyway it records applied changes in a forward-only history table and requires destructive changes to be explicit, and unlike either, it pairs this with PostgreSQL's transactional DDL and a pre-comparison advisory lock, so a failed migration is rolled back atomically and concurrent migrators cannot interleave.

2.5 Cross-Language Comparators

Several frameworks outside Java inform the design. Prisma, in TypeScript, uses a single declarative schema file from which a CLI generates a type-safe client and runs migrations, SDP4J borrows the commitment to a single source of truth but expresses it through Java annotations rather than a separate file. TypeORM is a decorator-driven ORM whose fluent query builder directly inspires SDP4J's high-level layer. SQLAlchemy, in Python, is the most important precedent for the two-layer design: it splits explicitly into a Core layer (a SQL expression language that can stand alone) and an ORM layer built on top, which is exactly the relationship between SDP4J's low-level and high-level layers. GORM, in Go, shows that struct-tag mapping and convention-over-configuration can produce a developer-friendly tool with a small public API, confirming the broader trend toward simpler, more transparent persistence.

3. Methodology

3.1 From Problem to Design Constraints

The methodology is the disciplined application of five design constraints derived from the problem of Chapter 1. First, the relational schema is authoritative: wherever a fact can live either in the database or in the framework, it lives in the database, and the framework only

makes it visible. In the vocabulary of enterprise patterns this is a commitment to the Data Mapper pattern over Active Record — the object is plain data, and all persistence behaviour lives in the framework around it. The practical consequence is one mapped class per table, with ordinary Data Transfer Objects used freely for any other view of the data.

Second, SQL is transparent: a developer who knows SQL should be able to predict the statement any call will produce and should never have to learn a second query language. This rules out a hidden query language and “magic” method-name parsing, builder methods are named after the SQL clauses they emit, and where a builder would be awkward the developer can drop one level down and write SQL directly. Third, execution is immediate and predictable: there is no persistence context, no dirty checking and no flush, so a statement runs when it is asked to and reading never schedules a later write. Fourth, validation is fail-fast at the boundary: because the framework lets developers write SQL fragments, every fragment is validated against the known schema before it is sent to the database, so a misspelled column is rejected in Java with a precise message. Fifth, a single tool covers mapping, migration and querying under one mental model, so adoption does not become the familiar exercise of assembling several libraries and learning the seams between them.

3.2 Technology and Platform Choices

The framework is written in Java and built with Maven, packaged as a single jar with a minimal dependency surface. It uses records for the immutable query and metadata model, switch expressions in the migration parser, and text blocks for the embedded catalog queries. Three runtime dependencies each fill exactly one role: HikariCP for connection pooling, the de-facto standard pool for the JVM, ClassGraph for the classpath scan that discovers annotated classes, which is genuinely fiddly to do well across class loaders and jar layouts, and the PostgreSQL JDBC driver, the only driver on the path, reflecting the deliberate single-dialect scope.

PostgreSQL is the initial and only target dialect, and the choice is methodological rather than incidental. PostgreSQL supports transactional DDL: CREATE TABLE, ALTER TABLE and DROP can all participate in a transaction and be rolled back. This is what makes the migration design safe without the framework implementing its own compensation logic — if any statement in a batch fails, the database itself unwinds every preceding statement in the same transaction. PostgreSQL also exposes a queryable set of system catalogs, which is

exactly the introspection surface the migration engine needs, and session-level advisory locks, which the engine uses to serialise concurrent migrators.

3.3 System Architecture

SDP4J is organised as a single client facade over three cooperating modules, plus a small core of shared utilities and a typed exception hierarchy. The client, `Sdp4jClient`, applies the Facade pattern: it is the only object the application constructs directly, it owns the `DataSource`, and it exposes the three modules through accessors. The connection pool it configures by default, HikariCP, is itself an instance of the Object Pool pattern, reusing a bounded set of physical connections across many logical operations. Figure 1 shows the arrangement.

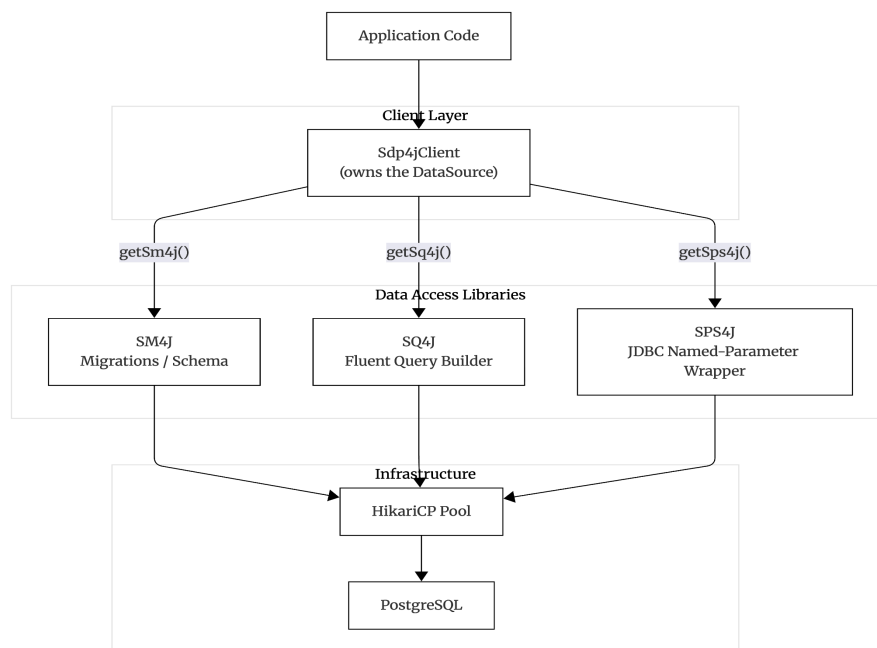


Figure 1. System architecture of SDP4J.

The three modules correspond to the three pillars. SM4J (Schema Migration for Java) owns the annotation-to-DDL mapping and the migration engine. SQ4J (Simple Query for Java) is the high-level, schema-validated fluent builder for SELECT, INSERT, UPDATE and DELETE. SPS4J (Simple Prepared Statement for Java) is the low-level layer, a thin named-parameter wrapper over a single JDBC `PreparedStatement` for cases that need full control of the SQL and the connection. SQ4J and SPS4J together form the two-layer query

interface, the layering deliberately echoes SQLAlchemy's Core/ORM split, where the lower layer is complete on its own and the higher layer adds convenience. The client can be constructed from a JDBC URL — in which case it configures a HikariCP pool with conservative defaults — or from a caller-supplied DataSource, for applications that manage pooling themselves, in both cases a package name tells the migration engine where to scan for mapped classes.

3.4 The Schema-First Data Model

A mapped class carries a `@Table` annotation, and each field carries annotations that name real SQL features. The set is intentionally small and one-to-one with SQL, summarised in Table 1. A short example conveys the style: the entity below declares a table, a primary key, two non-null columns (one with an explicit length) and a boolean column defaulting to false.

```
@Table(name = "users", mapToSnakeCase = true)
public class User {
    @PrimaryKey                private UUID id;
    @NotNull @Length(120)      private String firstName;
    @NotNull                   private String lastName;
    @DefaultFalse              private Boolean isActive;
    // getters and setters
}
```

Annotation	Target	Meaning
<code>@Table(name, mapToSnakeCase)</code>	class	Marks a mapped table; the name defaults to the class name (lower-cased, or snake-cased on request).
<code>@PrimaryKey</code>	field	Primary-key column (mandatory; every table must declare one).
<code>@NotNull</code>	field	NOT NULL constraint.
<code>@ForeignKey(mapsto, referencedColumn, action)</code>	field	Foreign key with an ON DELETE rule (RESTRICT, CASCADE, SET NULL, NO ACTION).
<code>@Index</code>	field	Creates a single-column index.
<code>@UniqueKeysConstraint(keys)</code>	class	Single- or multi-column unique constraint (repeatable).
<code>@Length(n)</code>	field	Sets VARCHAR(n); defaults to VARCHAR(255) when absent.
<code>@Default* (Int, BigInt, Real, DoublePrecision, Numeric, String, True, False)</code>	field	Typed column default, validated against the column's inferred SQL type.

Table 1. SDP4J mapping annotations.

Column types are not annotated separately, they are inferred from the Java field type according to the fixed mapping in Table 2, and any unmapped type is rejected at parse time. The typed default annotations are validated against the inferred type, so a numeric default on a textual column, for example, is a parse-time error rather than a latent bug. Naming is convention-driven and deterministic: table and column names derive from class and field names (optionally snake-cased), and generated objects follow readable patterns such as `fk_table_column`, `idx_table_column` and `uq_table_columns` — which matters because the migration engine must later find these objects by name when a rename cascades to them.

Java field type	PostgreSQL column type
String	VARCHAR(255), or VARCHAR(n) with @Length(n)
Integer / int	INT
Long / long	BIGINT
Float / float	REAL
Double / double	DOUBLE PRECISION
BigDecimal	NUMERIC
Boolean / boolean	BOOLEAN
LocalDate	DATE
LocalDateTime / Instant	TIMESTAMP
UUID	UUID

Table 2. Java-to-PostgreSQL type mapping.

The parser turns a class into an immutable description holding the table and its columns. This metadata model is the common language of the whole framework: the migration module reads it to generate DDL, and the query module wraps it (adding a column-to-field map) to drive validation and row mapping. Because there is a single parsing path, the schema the migrator builds and the schema the query layer validates against can never drift apart.

3.5 Migration Methodology

The migration engine is the most distinctive part of the design, and it works by separating two fundamentally different kinds of change. Additive changes are derived rather than written: because the mapped classes are the desired end state, anything that must be added to reach that state can be computed. On each run the engine scans the configured package for annotated classes, introspects the live database for its current tables and columns, and computes the difference — emitting `CREATE TABLE` for missing tables, `ALTER TABLE ...`

ADD COLUMN for missing columns, and the foreign keys and indexes implied by the annotations. This derivation is naturally idempotent: a class that already matches the database produces no statements.

Destructive and altering changes — dropping a column, renaming a table or column, changing a type, toggling a constraint — cannot be inferred safely from a class, because the class simply no longer mentions the thing. These are declared explicitly as an ordered list of steps obtained from helper methods, and each step records a human-readable action string that the engine stores in history to guarantee once-only application. The combined work is ordered in a dependency-safe sequence: create tables, add columns, apply the explicit steps, add foreign keys, add indexes — so that everything a foreign key or index references already exists.

Crucially, the entire operation — introspection, difference and application — runs on one connection inside a single transaction, and the engine acquires a PostgreSQL advisory lock as its very first action, before reading the database. This ordering is what makes concurrent migration safe: if two instances start together, the first to take the lock computes and applies its plan and commits, and only then does the second proceed, by which point its introspection sees the already-migrated schema and its difference is empty. Reading the database under the same lock and transaction that will apply the changes removes the time-of-check-to-time-of-use gap entirely. On success the transaction commits and writes one history row per step, on failure it rolls back, and because PostgreSQL DDL is transactional the schema is left exactly as it was, with an exception that names the failing step. Two safety rules are: the engine refuses to auto-add a NOT NULL column to an existing table unless it carries a default, and when a column rename is requested it can introspect the catalogs to rename the dependent index, foreign-key and unique constraints so the naming convention is preserved.

This design reflects a deliberate position on a problem that has no clean general solution. If migrations could be made safe, automatic and ergonomic all at once, the problem would already be solved; instead every existing tool ships a workaround — hand-written SQL files, an unsafe diff (Hibernate's auto-DDL cannot, for instance, distinguish a column rename from a drop-and-add), or paired up and down methods that are verbose and easy to get wrong. SDP4J's stance is to attempt no more than it can do safely. In an evolving, consistency-conscious schema, additive changes — new tables and columns — occur far more often than destructive ones, by a wide margin and often an order of magnitude or more;

the framework therefore automates exactly those, where the outcome is unambiguous, and asks the developer to write the comparatively rare destructive or altering changes by hand. This is also why the migration history is forward-only, in line with tools such as Flyway and with current industry practice: reversal is not automated because it cannot be automated reliably, so where a change must be undone the developer writes an explicit step for it. The trade-off is deliberately small, and it brings a further benefit — because destructive changes are written by hand and committed, version control becomes an accurate record of what changed and why.

Migrations are not run by a separate tool. The developer triggers them from an ordinary runnable entry point: typically a test class that applies and exercises the schema locally during development, and a small runner class invoked from the continuous-integration and deployment pipeline, which is where migrations are usually applied for production. The same entry point can be launched from the IDE's run or debug action or from the command line, so no additional tooling is imposed on the project.

3.6 Query Methodology

The two query layers are meant to be chosen per call rather than per project. The low-level layer, SPS4J, is in pattern terms an Adapter: it adapts a clean named-parameter interface onto JDBC's positional `PreparedStatement`. It replaces positional markers with named `:name` placeholders, binding every occurrence of a name from a single call, and it is literal- and cast-aware so that quoted text and the PostgreSQL `:: cast` are never mistaken for parameters. It translates checked exceptions into an unchecked form, and its staged builder is a Step Builder whose interface segregation makes it impossible at compile time to prepare a statement without first supplying a connection. The developer owns the connection's lifecycle, and an escape hatch returns the raw `PreparedStatement` when needed.

The high-level layer, SQ4J, is a fluent Builder whose method names mirror SQL and whose chain is realised as a sequence of single-method Step Builder interfaces, so the compiler guides the developer through the clauses in a sensible order. Every SQL fragment — both the WHERE clause and the join condition — uses the same named-parameter convention, so there is one binding style across the whole API. The builder does not concatenate SQL, it accumulates state and, on a terminal call, produces an immutable query object — in effect a small abstract syntax tree modelled as a Value Object using Java records. A dedicated

renderer then walks that tree and produces the final SQL together with an ordered binding list. Separating the representation from the operation that consumes it, in the spirit of the Interpreter/Visitor family, makes the builder both inspectable — the query can be previewed without executing it — and safe, because rendering is the only place SQL strings are assembled.

Validation happens before rendering. Column projections are checked against the tables in scope, and a bare column that is ambiguous across a join is rejected with a message asking the developer to qualify it. WHERE and JOIN fragments pass through a validator that distinguishes identifiers from reserved keywords, function calls, literals and casts, validates every column and qualifier, expands a collection bound to an IN parameter into the right number of placeholders, and enforces binding completeness. Execution and mapping are deliberately plain: an executor borrows a pooled connection, binds the values and runs the statement. Rows are mapped by a RowMapper, which is a Strategy with two concrete implementations chosen automatically by a small factory: an entity mapper for @Table classes and a forgiving DTO mapper for any other class, which matches columns to fields by name and ignores anything unmatched. Per-class descriptors are memoised so the reflection cost is paid once per type rather than once per query.

SQ4J deliberately does not pursue the compile-time type safety of a tool such as jOOQ, which achieves it through a code-generation step run against the schema. The framework trades that away for zero-setup ergonomics — a table is created by writing a class, which can then be queried immediately, with no generation step and no derived sources to keep in sync — and recovers much of the same protection by validating every column and binding against the schema at runtime, before the statement executes, so the same class of mistakes is still caught early rather than at the database.

3.7 End-to-End Solution Flow

The developer's workflow ties the modules together in a predictable arc, shown in Figure 2: model the tables as annotated classes, configure a client, run the migration (which derives the additive changes and applies them atomically), and then query through either layer, mapping results to entities or DTOs. The same annotated classes that define the schema also drive the migration and the query validation and mapping, with no separate schema file, no code-generation step and no second metadata source to keep in sync.

```
1. Model          write @Table-annotated classes (one per table)
```

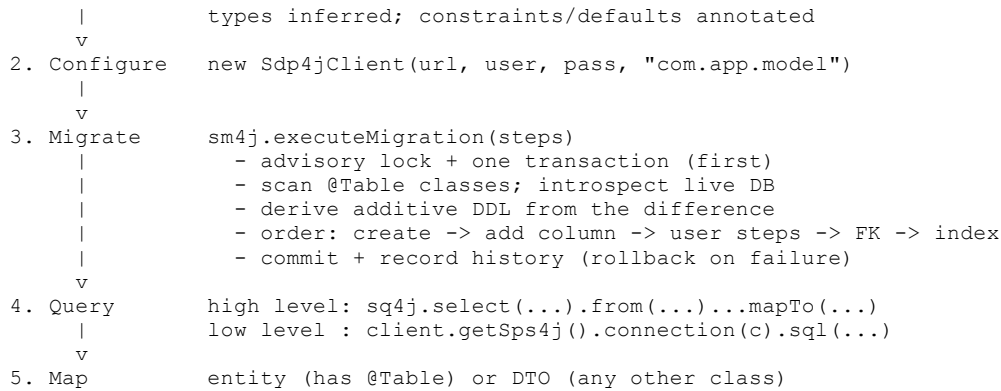


Figure 2. End-to-end developer workflow.

3.8 Data Acquisition and Handling

The framework works with two kinds of data obtained two different ways. Schema metadata is obtained by reflection over the mapped classes and by live introspection of the database through PostgreSQL's catalogs; these two views are compared to drive migration. Application data — the actual rows — flows only through JDBC: into the database via bound prepared-statement parameters, never string concatenation, which is the design's standing answer to SQL injection, and out of it via result-set reads that the mappers convert to objects by field type. Keeping these two paths cleanly separated is what lets the framework be confident about the schema while remaining uninvolved in the semantics of the data itself.

3.9 Validation Methodology

Because the project's success criteria are ergonomic rather than numerical — how little code, how few concepts, how predictable the result — the evaluation uses three complementary methods, all reported in Chapter 4. The first is an automated test suite with two tiers: a JUnit 5 unit corpus that exercises the metadata parser, the DDL generators, the migration-step model, the query builder and its validator, the mappers and the low-level wrapper using lightweight in-process JDBC fakes, so it runs deterministically without a database, and an integration suite that runs the framework end to end against a real PostgreSQL instance, verifying that DDL applies, that catalog types match the annotations, that values round-trip correctly and that concurrent migrations are safe. The second is functional validation through runnable examples that demonstrate every public capability of each layer against a real database. The third is a structural comparison of the same operations implemented in raw

JDBC, Hibernate and SDP4J. Runtime performance benchmarking is out of scope for this iteration and is identified as future work; the design targets developer effort and predictability, and the evaluation is scoped to match.

4. Implementation and Results

4.1 Project Structure and Build

The implementation is a single Maven module of roughly 5,300 lines of library Java, organised by responsibility: a core package (the client, shared utilities and the exception hierarchy), sm4j (annotations, metadata, DDL generators, the parser and the migration engine), sq4j (builders, step interfaces, query internals, renderers, validation, mappers and metadata caches), and sps4j (the named-parameter wrapper and its staged steps). Alongside the library, a demo package provides runnable usage examples, split into model classes and one runner per concern, sharing a single database configuration, and a test tree holds the automated suites. The build declares HikariCP, the PostgreSQL driver and ClassGraph as

runtime dependencies, and JUnit 5 and Testcontainers as test-scope dependencies. The sections below follow the order in which a developer meets the framework, quoting only the lines that carry the design.

4.2 The Client

Sdp4jClient is the composition root and the facade for the whole library. Its two constructors realise the two configuration paths: the connection-string constructor builds a HikariCP pool with conservative defaults — a small idle minimum, a modest maximum pool size, and finite connection, idle, lifetime and leak-detection timeouts — while the DataSource constructor hands pooling entirely to the application. The client wires the three modules and exposes them through accessors, and it provides a helper that closes a result set, any statements and a connection in safe order, swallowing secondary close failures so that one failure never leaks the rest. This keeps the low-level layer's call sites free of nested try-finally blocks.

4.3 Annotation Mapping

The annotations are deliberately declarative: most carry no members at all, the foreign-key annotation carries the referenced class, column and on-delete action, the length annotation carries a single integer, and the unique-constraint annotation is repeatable. A single metadata parser turns a class into an immutable table description. For each field it resolves the column type first — taking the length annotation into account for strings and mapping BigDecimal to NUMERIC — and then lets the annotations refine the column: marking the primary key, applying NOT NULL, attaching a typed default that is checked against the resolved type, and recording a foreign key whose target must itself be an annotated table. Because a VARCHAR may now carry a non-default length, the string-default check matches on the type prefix rather than an exact literal, so length and default compose cleanly.

4.4 DDL Generation

DDL is produced by a small family of generators sharing a common base that renders a single column and owns the deterministic naming helpers. The create generator assembles a CREATE TABLE ... IF NOT EXISTS from the columns, the mandatory primary key and any

table-level unique constraints, but deliberately omits foreign keys and indexes from the create statement, the add generator emits those separately as ALTER TABLE ... ADD CONSTRAINT and CREATE INDEX ... IF NOT EXISTS. Splitting table creation from foreign-key and index creation is what allows the engine to order a whole multi-table migration safely, because every table exists before any cross-table reference is added. Separate modify and drop generators produce the explicit alter, drop and rename statements behind the migration helper methods.

4.5 The Migration Engine

The migration entry point follows the methodology of Section 3.5 closely. The whole operation runs on one connection, inside one transaction, under the advisory lock taken as the first action, as the skeleton below shows.

```
try (Connection con = dataSource.getConnection()) {
    con.setAutoCommit(false);
    acquireMigrationLock(con);           // advisory lock, BEFORE any read
    if (!existsMigrationInDb(con)) { applyInitialSchema(con); con.commit();
return; }
    // introspect, diff against @Table classes, order, apply, record history
    con.commit();                       // rollback on any failure
}
```

Acquiring the transaction-scoped lock before introspection closes the time-of-check-to-time-of-use gap: two instances starting together serialise on the lock, and the second one's introspection — run after the first commits — sees the migrated schema, so its difference is empty. The initial run is a special case: when the history tables are absent the engine creates them, creates the entire schema implied by the classes, and records a single baseline step. The incremental run filters out explicit steps already in history, parses the remaining destructive steps to learn which tables and columns are being dropped or renamed, introspects the affected tables, and derives the additive work while taking renames and drops into account, so that a renamed column is not also re-added and a dropped table is skipped. On failure the engine decodes the batch-update exception to name the precise failing step.

4.6 The Low-Level Layer (SPS4J)

The substance of SPS4J is its named-parameter parser, which in a single pass rewrites each `:name` placeholder to a positional marker while recording the positions each name occupies, so that binding a name once fills every occurrence. The parser tracks whether it is inside a single- or double-quoted literal and recognises the PostgreSQL `:: cast`, so neither is mistaken for a parameter. Binding is available by name or by raw position, with explicit null binding and batch support, and an escape hatch exposes the underlying statement. The staged builder enforces, at compile time, that a connection is supplied before any SQL is prepared, and closing a handle closes only its statement — never the caller's connection — so the adapter never assumes ownership of a resource it did not create.

4.7 The High-Level Layer (SQ4J)

The SQ4J facade exposes `select`, `insertInto`, `update` and `deleteFrom` over an executor and two metadata caches. The builders implement the chain of single-method step interfaces that gives the fluent API its guided, hard-to-misuse shape: at each point only the methods that make sense next are available. A terminal call produces an immutable query record holding the projected fields, the source table, the joins, the rendered WHERE clause and its bindings, the ordering and the paging, a renderer then turns that record into SQL and an ordered binding list. Every WHERE and ON fragment uses the same named-parameter convention, so the four statement types read identically. Projection validation rejects an unknown qualifier, an unknown column and an ambiguous bare column before any SQL is sent, and the shared validator performs the same checks, the collection expansion and the binding-completeness checks for every fragment. Mapping is reflective and descriptor-driven, with the entity or DTO mapper chosen automatically by testing for the table annotation.

4.8 Validation and Results

This section evaluates SDP4J against the goals set out in Chapter 1. Because the framework's value is ergonomic rather than performance-related, the evaluation deliberately does not benchmark execution speed: a thin wrapper over JDBC would simply track raw JDBC, and a feature-by-feature speed contest with a caching ORM such as Hibernate would be misleading. Instead it compares behaviour and transparency against other persistence tools,

tabulates the conceptual surface each imposes on the developer, demonstrates the migration engine’s safety with a concrete experiment, and reports the automated test coverage.

4.8.1 Behavioural comparison with other tools

Rather than counting lines of code — a brittle, style-dependent metric — the comparison focuses on behaviour and transparency: what each tool makes the developer think about, and what it hides. Each snippet performs the same task: fetch the active users named “Alice”.

SDP4J

```
List<User> users = sq4j.select("").from(User.class)
    .where("is_active = :active AND first_name = :name")
    .set(":active", true)
    .set(":name", "Alice")
    .mapTo(User.class);
```

The statement is plain SQL, nothing runs until `mapTo(...)`; the column names are validated against the schema before the query is sent.

Hibernate / JPA (JPQL)

```
List<User> users = em.createQuery(
    "FROM User u WHERE u.active = :active AND u.firstName = :name", User.class)
    .setParameter("active", true)
    .setParameter("name", "Alice")
    .getResultList();
```

JPQL is a second query language layered over the entities rather than SQL. The returned objects are attached to the persistence context, so a later field change can be flushed to the database with no explicit save, and traversing an unfetched association can throw `LazyInitializationException` once the session is closed.

Spring Data JPA

```
List<User> users = userRepository.findByIsActiveTrueAndFirstName("Alice");
```

The query is generated from the method name; the developer must learn the naming grammar, the SQL is invisible, and a malformed name fails at application startup rather than where it is written.

jOOQ

```
List<User> users = create.selectFrom(USERS)
    .where(USERS.IS_ACTIVE.eq(true).and(USERS.FIRST_NAME.eq("Alice")))
    .fetchInto(User.class);
```

Type-safe and faithful to SQL, but USERS and its columns are generated classes produced by a build-time code-generation step run against a live schema — an extra moving part SDP4J does not require.

MyBatis (mapper interface + XML)

```
List<User> findActiveByName(@Param("name") String name);
```

```
<select id="findActiveByName" resultType="User">
  SELECT * FROM users WHERE is_active = true AND first_name = #{name}
</select>
```

Honest about SQL, but the statement lives in a separate XML file (or a long annotation string), away from the call site.

Across these tools, SDP4J keeps the SQL visible and the execution immediate, needs no code-generation step and no second query language, and validates against the schema before running — trading away jOOQ’s compile-time type safety and Spring Data’s method-name convenience in exchange for transparency and zero setup.

A second illustration is the “hidden write”. In Hibernate:

```
User u = em.find(User.class, id);
u.setFirstName("Alice");    // no save() - dirty checking flushes an UPDATE at
                             commit
```

the row is updated with no explicit instruction.

In SDP4J the same change is explicit and nothing is written unless execute() is called:

```
User patch = new User();
patch.setFirstName("Alice");
sq4j.update(User.class).set(patch).where("id = :id").set(":id", id).execute();
```

4.8.2 Conceptual surface

The clearest expression of “less to learn” is the set of distinct concepts a developer must hold to perform basic CRUD.

Tool	Concepts the developer must understand
Raw JDBC	Connection, PreparedStatement, ResultSet, positional parameter indexing, manual row-to-object mapping, resource closing / try-with-resources, transaction & commit
Hibernate / JPA	persistence context, entity lifecycle (attached/detached), EntityManager, dirty checking & flush modes, lazy loading & proxies, fetch types, cascade types, first/second-level cache, JPQL/Criteria
Spring Data JPA	everything in JPA, plus the derived-query method-name grammar and the repository abstraction
jOOQ	the generated schema classes, the code-generation build step, and the DSL
MyBatis	mapper interfaces, XML mapper files (or annotation SQL), result maps, parameter syntax
SDP4J	a client object, the mapping annotations, a fluent builder whose methods mirror SQL — and SQL itself (jdbc for using SPS4J)

Table 3. Concepts required for basic CRUD across persistence tools.

SDP4J’s row is short by design: it introduces no persistence context, no second query language, no code-generation step, and no separate mapping file.

4.8.3 Migration-safety experiment

This experiment validates the central claim of the migration engine — that it handles a structural change safely where the dominant automatic approach cannot. The scenario: a users table populated with 1,000 rows, in which the column fname must be renamed to first_name (the developer renames the field fname to firstName in the mapped class).

Under Hibernate with hbm2ddl.auto=update, Hibernate compares the entity to the database, sees a field first_name with no matching column and a column fname that is no longer mapped. Because update only ever adds, it issues ALTER TABLE users ADD COLUMN first_name ... and leaves fname untouched: the result is a new, empty first_name column

while the data stays stranded in the now-unmapped fname, so the application reads nulls and a manual data-copy script is needed. Under create or create-drop the table is dropped and recreated, losing all 1,000 rows outright. The underlying limitation is that a diff cannot distinguish a rename from a drop-plus-add.

Under SDP4J, the developer states the rename explicitly:

```
sm4j.renameColumn("users", "fname", "first_name", true); // true: also rename
dependent objects
sm4j.executeMigration(steps);
```

This emits ALTER TABLE users RENAME COLUMN fname TO first_name inside the migration transaction and, because the dependent flag is set, also renames the column's index, foreign-key and unique constraints to match the naming convention. All 1,000 rows are preserved.

Approach	DDL issued	Rows preserved	Manual fix needed
Hibernate (update)	ADD COLUMN first_name	0 — data stranded in fname	Yes
Hibernate (create)	DROP + CREATE TABLE	0 — all rows lost	Yes
SDP4J	RENAME COLUMN fname TO first_name	1,000 — all preserved	No

Table 4. Outcome of renaming a column on a populated table.

The safety comes precisely from SDP4J refusing to infer destructive intent: it automates only additive changes and requires the rename to be written by hand, which also leaves a record in version control of what changed and why.

4.8.4 Test coverage

Correctness is validated by two automated tiers plus runnable examples. The unit tier (35 tests across 8 classes) runs with no database, faking JDBC with dynamic proxies so it can assert the exact SQL and bindings a builder produces and the objects a row maps to.

Unit test class	Verifies
MetadataParserAndDdlTest	annotation parsing, type inference, DDL generation
DdlGeneratorsAndMigrationStepsTest	DDL generators and the migration-step model

Sq4jQueryBuilderTest	fluent builder rendered to SQL and bindings
Sq4jValidationEdgeCasesTest	rejection of unknown/ambiguous columns and bad bindings
QueryExecutorAndMetadataTest	execution path and metadata caches
MappingAndSps4jTest	row mapping and the low-level wrapper
JdbcValueReaderTest	type-correct value reading
CoreUtilitiesAndClientTest	client and shared utilities

Table 5. Unit-test coverage (35 tests, no database).

The integration tier (17 tests) runs end to end against a real PostgreSQL, supplied by Testcontainers or an external database and skipped via a JUnit assumption when neither is available.

Integration test class	Verifies
MigrationIntegrationTest	schema creation and additive evolution apply correctly
ConcurrentMigrationIntegrationTest	two simultaneous migrators create the schema exactly once
Sq4jIntegrationTest	end-to-end SELECT/INSERT/UPDATE/DELETE, joins, DTO mapping
Sps4jIntegrationTest	the low-level named-parameter path end to end
PostgresIntegrationTest	shared base: schema reset and connectivity

Table 6. Integration-test coverage (17 tests, real PostgreSQL).

The concurrency test is the most important single case: it turns the lock-first migration design into a verified property rather than a claim.

4.9 Discussion

Taken together, the results support the central design bet. The migration engine removes the most tedious half of schema management — writing additive DDL — entirely, while keeping the genuinely risky half explicit and recorded and PostgreSQL's transactional DDL, combined with the lock-first, single-transaction execution, makes the whole operation atomic

and safe under concurrency without the framework simulating rollback itself. The integration test that starts two migrators at once turns this from a design claim into a verified property. The two query layers let a developer stay high-level for the common cases and drop one level when a statement needs hand-shaping, without leaving the framework or learning a new vocabulary, and the single named-parameter convention now runs uniformly across both. The fail-fast validation converts a whole class of runtime SQL errors into immediate, well-located Java errors, which is the concrete mechanism by which “transparent” becomes “safe” rather than merely “exposed.” Where the design trades away breadth — a single dialect, and no object yet for grouping several statements into one application-level transaction — those are scoped, visible trade-offs rather than hidden complexity, and each is revisited in Chapter 5. The clearest threat to validity is that the structural comparison is qualitative; a future quantitative study, both of code volume measured systematically and of runtime performance, would strengthen the conclusions.

5. Conclusions

5.1 Achievements

This thesis set out to test a specific hypothesis: that for a developer who already knows Java and SQL, a thin and visible bridge across the object-relational gap is more useful, on small and medium projects, than a thick abstraction that hides the relational model. The work produced a complete, working framework that embodies that bridge and allows the hypothesis to be examined against a real artifact rather than an argument. The mapping layer turns annotated classes into PostgreSQL schema with a small vocabulary that names real SQL features and infers column types from Java types. The migration engine contributes the design's most original idea — deriving additive changes automatically while requiring destructive ones to be explicit, and applying them atomically and concurrency-safely on transactional DDL with an advisory lock and a history table. The two-layer query interface

provides a low-level named-parameter wrapper that removes JDBC's boilerplate without hiding its SQL and a high-level fluent builder that mirrors SQL, validates against the schema before execution and maps to entities or DTOs.

An automated test suite — unit tests backed by in-process JDBC fakes, plus an integration suite that runs end to end against a real PostgreSQL, including the regression test that proves the migration engine is concurrency-safe — confirms that the framework behaves as specified, including the deliberate rejections, and a package of runnable examples exercises each layer against a live database. The structural comparison shows the predicted outcome. In short, the central claim of Chapter 1 is realised in the built system: the framework introduces no third large concept on top of Java and SQL, its annotations are SQL features, its query methods are SQL clauses, and its execution model is simply that a statement runs when it is run.

5.2 Limitations

The framework targets a single dialect and relies on PostgreSQL-specific facilities, including transactional DDL, advisory locks, and the system catalogs. As a result, supporting additional database systems would require a dialect-abstraction layer that does not yet exist.

There is also no application-level transaction object. Because each high-level call borrows its own pooled connection, multiple SQ4J operations cannot yet be grouped into a single atomic unit of work without using the low-level layer and managing the connection manually.

The set of supported column types is currently limited and fixed. There is no support for arrays, JSON, or timezone-aware timestamps, meaning that schemas requiring such types cannot be fully expressed through the framework at present.

The evaluation also has limitations. The comparison presented in Chapter 4 is behavioural and structural rather than based on a controlled user study, and the example operations were selected and implemented by the author. Consequently, the ergonomic claims, while concrete and reproducible, are not statistically established.

These limitations represent clear boundaries of the current version rather than accidental omissions. The most significant areas for future development, particularly database-dialect

abstraction and application-level transaction management, are discussed in the following section.

5.3 Future Work

Most future items are extensions rather than redesigns, which is itself evidence that the architecture is sound. One direction is a performance benchmark comparing SDP4J with raw JDBC, Hibernate and jOOQ on equivalent operations, which would turn the structural comparison of Chapter 4 into a measured one. Beyond these, an explicit transaction handle would let several high-level calls share one connection and commit or roll back together, closing the one real gap on the write path while preserving the no-dirty-checking philosophy, broader type and precision coverage — configurable numeric precision, timezone-aware timestamps, arrays and JSON — would widen the range of expressible schemas, and, looking further out, the engine's reliance on PostgreSQL-specific catalogs suggests a pluggable dialect layer as the path to a second database, while the precedent of compile-time data access suggests moving some of the reflection-driven work to annotation processing for faster startup and earlier error detection. None of these directions requires abandoning the core position: they extend a thin, visible bridge rather than thickening it.

6. Bibliography

- [1] M. Tyson, “What is JDBC? Introduction to Java Database Connectivity,” InfoWorld, <https://www.infoworld.com/article/2260118/what-is-jdbc-introduction-to-java-database-connectivity.html> (accessed April 2026).
- [2] G. King, “Hibernate ORM 6.6 User Guide,” Hibernate Project, https://docs.hibernate.org/orm/6.6/userguide/html_single/ (accessed April 2026).
- [3] V. Mihalcea, (2016), “High-Performance Java Persistence,” self-published, Amazon KDP.
- [4] V. Mihalcea, “The best way to handle the LazyInitializationException,” <https://vladmihalcea.com/the-best-way-to-handle-the-lazyinitializationexception/> (accessed April 2026).
- [5] T. Neward, “The Vietnam of Computer Science,” Neward & Associates blog, <https://blogs.newardassociates.com/blog/2006/the-vietnam-of-computer-science.html> (accessed April 2026).

- [6] J. Atwood, “Object-Relational Mapping is the Vietnam of Computer Science,” Coding Horror, <https://blog.codinghorror.com/object-relational-mapping-is-the-vietnam-of-computer-science/> (accessed April 2026).
- [7] L. Eder, “jOOQ vs. Hibernate: When to Choose Which,” Java, SQL and jOOQ Blog, <https://blog.jooq.org/jooq-vs-hibernate-when-to-choose-which/> (accessed April 2026).
- [8] L. Eder, “10 Common Mistakes Java Developers Make when Writing SQL,” Java, SQL and jOOQ Blog, <https://blog.jooq.org/10-common-mistakes-java-developers-make-when-writing-sql/> (accessed April 2026).
- [9] Data Geekery GmbH, “jOOQ User Manual,” <https://www.jooq.org/learn/> (accessed April 2026).
- [10] MyBatis Project, “MyBatis 3 Reference Documentation,” <https://mybatis.org/mybatis-3/> (accessed April 2026).
- [11] Jdbi Contributors, “Jdbi 3 Developer Guide,” <https://jdbi.org/> (accessed April 2026).
- [12] Redgate, “Flyway Concepts: Versioned Migrations,” <https://documentation.red-gate.com/fd/versioned-migrations-273973333.html> (accessed April 2026).
- [13] Liquibase Inc., “Liquibase Documentation,” <https://docs.liquibase.com/> (accessed April 2026).
- [14] PostgreSQL Global Development Group, “Transactional DDL in PostgreSQL: A Competitive Analysis,” PostgreSQL Wiki, https://wiki.postgresql.org/wiki/Transactional_DDL_in_PostgreSQL (accessed April 2026).
- [15] TypeORM, “TypeORM Documentation,” <https://typeorm.io/> (accessed April 2026).
- [16] J. Schauder, “Introducing Spring Data JDBC,” Spring Engineering Blog, <https://spring.io/blog/2018/09/17/introducing-spring-data-jdbc/> (accessed April 2026).
- [17] G. Long, “Announcing Micronaut Data,” Micronaut Framework Blog, <https://micronaut.io/2019/07/18/announcing-micronaut-data/> (accessed April 2026).
- [18] Prisma, “What is Prisma ORM? (Overview),” Prisma Documentation, <https://www.prisma.io/docs/orm> (accessed April 2026).
- [19] C. Bauer, G. King and G. Gregory, (2015), “Java Persistence with Hibernate,” 2nd ed., Manning Publications, Shelter Island, NY.
- [20] Eclipse Foundation, (2022), “Jakarta Persistence 3.1 Specification,” <https://jakarta.ee/specifications/persistence/3.1/> (accessed April 2026).
- [21] Querydsl Contributors, “Querydsl Reference Guide,” <http://querydsl.com/> (accessed April 2026).
- [22] R. Bygrave, “Ebean ORM Documentation,” <https://ebean.io/docs/> (accessed April 2026).
- [23] M. Bayer, “SQLAlchemy 2.0 Released,” SQLAlchemy Blog, <https://www.sqlalchemy.org/blog/2023/01/26/sqlalchemy-2.0.0-released/> (accessed April 2026).
- [24] GORM, “GORM Guides,” <https://gorm.io/docs/> (accessed April 2026).